

EXP 5

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts.

Theory:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility defines who can access a function:**
 - **public:** available both inside and outside the contract.
 - **private:** only accessible within the same contract.
 - **internal:** accessible within the contract and its child contracts.
 - **external:** can be called only by external accounts or other contract
- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).

- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

4. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays:** Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings:** Key-value pairs that allow quick lookups. Example: mapping(address=> uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs:** Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums:** Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage:** Data stored permanently on the blockchain. Examples: state variables.
- **memory:** Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata:** A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

OUTPUT:

Tutorial 1:

The image displays three sequential screenshots of the Remix IDE interface, illustrating the process of deploying and interacting with a Solidity contract named 'Counter'.

Top Screenshot: The 'DEPLOY & RUN TRANSACTIONS' panel on the left shows the 'Environment' tab with 'Remix VM' selected. The 'Deployed Contracts' section lists the 'Counter' contract at address 0xd91...39138. The 'Functions' list includes 'dec', 'inc', 'count', and 'get'. The 'Code' editor on the right shows the contract source code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) {
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public {
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public {
19        count -= 1;
20    }
21 }
```

The 'Explain contract' panel at the bottom shows the first transaction: '[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20833 logs: 0 hash: 0xc1b...b5b21'.

Middle Screenshot: The 'Deployed Contracts' section now shows the 'count' function being called. The 'Functions' list is updated to show 'count' as the active function. The 'Code' editor shows the same contract code.

The 'Explain contract' panel shows the second transaction: '[vm] from: 0x5B3...eddC4 to: Counter.(inc()) 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0xe28...23b5d'.

Bottom Screenshot: The 'Deployed Contracts' section shows the 'count' function being called again. The 'Functions' list is updated to show 'count' as the active function. The 'Code' editor shows the same contract code.

The 'Explain contract' panel shows the third transaction: '[vm] from: 0x5B3...eddC4 to: Counter.(inc()) 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0xe99...5ae56'.

Tut 2:

The screenshot shows the Learneth Solidity Beginner Course interface. On the left, the 'Tutorials list' shows '2. Basic Syntax' (2/19) as the current tutorial. The main content area explains variable visibility and public variables, followed by an 'Assignment' with four tasks: deleting HelloWorld, creating a contract named 'MyContract', adding a public state variable 'name' of type 'string', and assigning the value 'Alice'. At the bottom, a 'Check Answer' button is active, and a green message says 'Well done! No errors.' On the right, the 'Create a DApp with your contracts' panel shows the Solidity code for 'MyContract' and a successful compilation log.

```
// SPDX-License-Identifier: MIT
// compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
pragma solidity ^0.8.3;

contract MyContract {
    string public name = "Alice";
}
```

Compilation log: [vm] from: 0x583...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xc1b...b5b21

Tut 3:

The screenshot shows the Learneth Solidity Beginner Course interface for Tutorial 3: Primitive Data Types (3/19). The main content area explains default values for data types and lists primitive types: Fixed Point Numbers, Byte Arrays, Strings, Mappings, Arrays, Enums, and Structs. The 'Assignment' has three tasks: creating a variable 'newAddr' of type 'address', a variable 'neg' of type 'int', and a variable 'newU' of type 'uint'. A tip suggests looking at the 'addr' variable in the previous contract. On the right, the 'Create a DApp with your contracts' panel shows the Solidity code for 'primitiveDataTypes.sol' and a successful compilation log.

```
address public newAddr = 0x321f6873b6dFd5C73731C39Fd9C89c7788D5EBc;
int public neg = -13;
uint8 public newU = 0;
```

Compilation log: [vm] from: 0x583...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xc1b...b5b21

Tut 4:

LEARNETH 2.6.2 Learneth Solidity Beginner Course

4. Variables 4 / 19

to be declared but can be accessed from within your contract. Global Variables are used to retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the Solidity documentation.

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ **Assignment**

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
4 contract Variables {
5   // State variables are stored on the blockchain.
6   string public text = "Hello";
7   uint public num = 123;
8   uint public blockNumber;
9
10  function doSomething() public { 22334 gas
11    // local variables are not saved to the blockchain.
12    uint i = 456;
13
14    // Here are some global variables
15    uint timestamp = block.timestamp; // Current block timestamp
16    address sender = msg.sender; // address of the caller
17    blockNumber = block.number;
18  }
19 }
```

Create a DApp with your contracts Start now

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Tut 5:

LEARNETH 2.6.2 Learneth Solidity Beginner Course

5.1 Functions - Reading and Writing to a State Variable 5 / 19

To define a function, use the `function` keyword followed by a unique name.

If the function takes inputs like our `set` function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on [Functions](#).

★ **Assignment**

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5   // State variable to store a number
6   uint public num;
7   bool public b = true;
8
9   // You need to send a transaction to write to a state variable.
10  function set(uint _num) public { 22536 gas
11    num = _num;
12  }
13
14  // You can read from a state variable without sending a transaction.
15  function get() public view returns (uint) { 2475 gas
16    return num;
17  }
18
19  function get_b() public returns (bool) { 2539 gas
20    return b;
21  }
22 }
```

Create a DApp with your contracts Start now

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Tut 6:

The screenshot shows the Remix IDE interface. On the left, the 'LEARNETH' sidebar displays the tutorial '5.2 Functions - View and Pure' (6 / 19). The main content area shows the tutorial text, which explains the difference between pure and view functions. It states that pure functions do not modify the state, while view functions do not read or modify the state. The tutorial includes an assignment: 'Create a function called addToX2 that takes the parameter y and updates the state variable x with the sum of the parameter and the state variable x.' Below the assignment, there are buttons for 'Check Answer' and 'Show answer', and a 'Next' button. The bottom status bar shows 'Well done! No errors.'

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) { infinite gas
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) { infinite gas
14        return i + j;
15    }
16
17    function addToX2(uint y) public returns (uint) { infinite gas
18        x = x + y;
19    }
20 }
```

Tut 7:

The screenshot shows the Remix IDE interface. On the left, the 'LEARNETH' sidebar displays the tutorial '5.3 Functions - Modifiers and Constructors' (7 / 19). The main content area shows the tutorial text, which explains the difference between pure and view functions. It states that pure functions do not modify the state, while view functions do not read or modify the state. The tutorial includes an assignment: 'Create a new function, increaseX, in the contract. The function should take an input parameter of type uint and increase the value of the variable x by the value of the input parameter. Make sure that x can only be increased. The body of the function increaseX should be empty. Tip: Use modifiers.' Below the assignment, there are buttons for 'Check Answer' and 'Show answer', and a 'Next' button. The bottom status bar shows 'Well done! No errors.'

```
41
42
43 locked = true;
44 _;
45 locked = false;
46
47
48 modifier biggerThan0(uint y) {
49     requires(y > 0, "Not bigger than x");
50     _;
51 }
52
53 modifier increaseXbyY(uint y) {
54     _;
55     x = x + y;
56 }
57
58 function increaseX(uint y) public onlyOwner biggerThan0(y) increaseXbyY(y) { infinite gas
59 }
60
61 function decrement(uint i) public noReentrancy { infinite gas
62     x -= i;
63
64     if (i > 1) {
65         decrement(i - 1);
66     }
67 }
```

Tut 8:

LEARNETH

Tutorials list

Learneth Solidity Beginner Course/4. Variables/variables.sol

5.4 Functions - Inputs and Outputs 8 / 19

5.4 Functions - Inputs and Outputs

In this section, we will learn more about the inputs and outputs of functions.

Multiple named Outputs

Functions can return multiple values that can be named and assigned to their name.

The `returnMany` function (line 6) shows how to return multiple values. You will often return multiple values. It could be a function that collects outputs of various functions and returns them in a single function call for example.

The `named` function (line 19) shows how to name return values. Naming return values helps with the readability of your contracts. Named return values make it easier to keep track of the values and the order in which they are returned. You can also assign values to a name.

The `assigned` function (line 33) shows how to assign values to a name. When you assign values to a name you can omit (leave out) the return statement and return them individually.

Deconstructing Assignments

You can use deconstructing assignments to unpack values into distinct variables.

The `destructuringAssignments` function (line 49) assigns the values of the `returnMany` function to the new local variables `i`, `b`, and `j` (line 60).

Input and Output restrictions

There are a few restrictions and best practices for the input and output parameters of contract

```
72
73 // Can use array for output
74 uint[] public arr;
75
76 function arrayOutput() public view returns (uint[] memory) {
77     return arr;
78 }
79
80 function returnTwo()
81     public
82     pure
83     returns (
84         int i,
85         bool b
86     )
87 {
88     i = -2;
89     b = true;
90 }
91 }
```

0 Listen on all transactions Filter with transaction hash or address

Tut 9:

LEARNETH

Tutorials list

Learneth Solidity Beginner Course/4. Variables/variables.sol

6. Visibility 9 / 19

external

- Can be called from other contracts or transactions
- State variables can not be `external`

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on Visibility.

Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer Show answer

Next

Well done! No errors.

```
51 // State variables cannot be external so this code won't compile.
52 // string external externalVar = "my external variable";
53 }
54
55 contract Child is Base {
56     function testInternalFunc() public pure override returns (string memory) {
57         return internalFunc();
58     }
59
60     function testInternalVar() public view returns (string memory, string memory) {
61         return (internalVar, publicVar);
62     }
63 }
```

0 Listen on all transactions Filter with transaction hash or address

Tut 10:

The screenshot shows the LearnNETH Solidity tutorial interface for Tut 10. The left sidebar contains a navigation menu with options like AI, Files, Search, Compile, Deploy, Debug, Solidity, and Frontend. The main content area displays the tutorial text for '7.1 Control Flow - If/Else', explaining the 'else if' statement. Below the text is an assignment section titled 'Assignment' with instructions to create a function 'evenCheck' in the 'IfElse' contract. The assignment includes a list of requirements: taking a 'uint' as an argument, returning 'true' for even and 'false' for odd, and using a ternary operator. A 'Check Answer' button and a 'Show answer' button are visible. The bottom status bar shows 'Well done! No errors.' and a 'Next' button.

LEARNETH

Tutorials list

LearnNETH Solidity Beginner Course/4. Variables/Variables.sol

7.1 Control Flow - If/Else

10 / 19

In this contract, the `foo` function uses the `else` statement (line 10) to return `2` if none of the other conditions are met.

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

★ **Assignment**

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer Show answer

Next

Well done! No errors.

Create a DApp with your contracts Start now

```
24
25 function evenCheck(uint y) public pure returns (bool) { Infinite gas
26     return y%2 == 0 ? true : false;
27 }
28
29
```

AI Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Tut 11:

The screenshot shows the LearnNETH Solidity tutorial interface for Tut 11. The left sidebar contains a navigation menu with options like AI, Files, Search, Compile, Deploy, Debug, Solidity, and Frontend. The main content area displays the tutorial text for '7.2 Control Flow - Loops', explaining the 'continue' and 'break' statements. Below the text is an assignment section titled 'Assignment' with instructions to create a function 'loop' in the 'Loop' contract. The assignment includes a list of requirements: creating a public 'uint' state variable called 'count', incrementing the count variable by 1 at the end of the for loop, and trying to get the count variable to be equal to 9. A 'Check Answer' button and a 'Show answer' button are visible. The bottom status bar shows 'Well done! No errors.' and a 'Next' button.

LEARNETH

Tutorials list

LearnNETH Solidity Beginner Course/4. Variables/Variables.sol

7.2 Control Flow - Loops

11 / 19

The `continue` statement is used to skip the remaining code block and start the next iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second if statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the `break` statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

★ **Assignment**

- Create a public `uint` state variable called `count` in the `Loop` contract.
- At the end of the for loop, increment the count variable by 1.
- Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer Show answer

Next

Well done! No errors.

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Loop {
5
6     uint public count;
7
8     function loop() public { Infinite gas
9         // for loop
10         for (uint i = 0; i < 10; i++) {
11             if (i == 3) {
12                 // skip to next iteration with continue
13                 continue;
14             }
15
16             count++;
17
18             if (i == 9) {
19                 // Exit loop with break
20                 break;
21             }
22         }
23     }
24 }
25 // while loop
```

AI Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Tut 12:

The screenshot shows the Learneth Solidity Beginner Course interface for Tut 12, titled "Arrays". The left sidebar contains a navigation menu with options like "Files", "Search", "Compile", "Deploy", "Debug", "Solidity", and "Frontend". The main content area on the left provides an explanation of the `delete` operator and the `length` property of arrays, followed by an "Assignment" section with two tasks: 1. Initialize a public fixed-sized array called `arr3` with values 0, 1, 2. 2. Change the `getArr()` function to return the value of `arr3`. Below the assignment are "Check Answer" and "Show answer" buttons, and a "Next" button. A green status bar at the bottom says "Well done! No errors." The right sidebar shows a "Create a DApp with your contracts" button and a "Start now" button. The main code editor displays the following Solidity code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Array {
5     // Several ways to initialize an array
6     uint[] public arr;
7     uint[] public arr2 = [1, 2, 3];
8     // Fixed sized array, all elements initialize to 0
9     uint[10] public myFixedSizeArr;
10    uint[3] public arr3 = [0, 1, 2];
11
12    function get(uint i) public view returns (uint) {
13        return arr[i];
14    }
15
16    // Solidity can return the entire array,
17    // but this function should be avoided for
18    // arrays that can grow indefinitely in length.
19    function getArr() public view returns (uint[3] memory) {
20        return arr3;
21    }
22
23    function push(uint i) public {
24        // Append to array
25        // This will increase the array length by 1.
26    }
```

Tut 13:

The screenshot shows the Learneth Solidity Beginner Course interface for Tut 13, titled "Mappings". The left sidebar contains a navigation menu with options like "Files", "Search", "Compile", "Deploy", "Debug", "Solidity", and "Frontend". The main content area on the left provides an explanation of the `mapping` type and the `delete` operator, followed by an "Assignment" section with three tasks: 1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`. 2. Change the functions `get` and `remove` to work with the mapping `balances`. 3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter. Below the assignment are "Check Answer" and "Show answer" buttons, and a "Next" button. A green status bar at the bottom says "Well done! No errors." The right sidebar shows a "Create a DApp with your contracts" button and a "Start now" button. The main code editor displays the following Solidity code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Mapping {
5     // Mapping from address to uint
6     mapping(address => uint) public balances;
7
8     function get(address _addr) public view returns (uint) {
9         // Mapping always returns a value.
10        // If the value was never set, it will return the default value.
11        return balances[_addr];
12    }
13
14    function set(address _addr) public {
15        // Update the value at this address
16        balances[_addr] = _addr.balance;
17    }
18
19    function remove(address _addr) public {
20        // Reset the value to the default value.
21        delete balances[_addr];
22    }
23 }
24
25 contract NestedMapping {
```

Tut 14:

The screenshot shows the Learneth Solidity Beginner Course interface. The left sidebar contains a search bar, a syllabus, and a list of topics. The main content area displays the 'Accessing structs' section, which includes a video tutorial and an assignment. The assignment asks the user to create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping. The right sidebar shows a code editor with the following Solidity code:

```
32 // Create a DApp with your contracts
33 // Update text
34 // Update completed
35 // Remove a struct member
36
37 // Update text
38 function update(uint _index, string memory _text) public {
39     // Update text
40     todos[_index].text = _text;
41 }
42
43 // Update completed
44 function toggleCompleted(uint _index) public {
45     // Toggle completed
46     todos[_index].completed = !todos[_index].completed;
47 }
48
49 function remove(uint _index) public {
50     delete todos[_index];
51 }
```

Tut 15:

The screenshot shows the Learneth Solidity Beginner Course interface. The left sidebar contains a search bar, a syllabus, and a list of topics. The main content area displays the 'Updating an enum value' section, which includes a video tutorial and an assignment. The assignment asks the user to define an enum type called `Size` with members `S`, `M`, and `L`, initialize the variable `sizes` of the enum type `Size`, and create a getter function `getSize()` that returns the value of the variable `sizes`. The right sidebar shows a code editor with the following Solidity code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Enum {
5     // Enum representing shipping status
6     enum Status {
7         Pending,
8         Shipped,
9         Accepted,
10        Rejected,
11        Canceled
12    }
13
14     enum Size {
15         S,
16         M,
17         L
18     }
19
20     // Default value is the first element listed in
21     // definition of the type, in this case "Pending"
22     Status public status;
23     Size public sizes;
24
25     function get() public view returns (Status) {
26         return status;
27     }
28 }
```

Tut 16:

LEARNETH 2.6.2 Learneth Solidity Beginner Course

9. Data Locations 16 / 19

If we were to create a new struct `myMemStruct3` with the data location `memory` inside the `function f` (line 12) and assign it the value of `myStruct`, changes in `myMemStruct3` would not affect the values stored in the mapping `myStructs` (line 10).

As we said in the beginning, when creating contracts we have to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

★ Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer **Show answer**

Next

Well done! No errors.

```
3
4 contract DataLocations {
5   uint[] public arr;
6   mapping(uint => address) map;
7   struct MyStruct {
8     uint foo;
9   }
10  mapping(uint => MyStruct) public myStructs;
11
12  function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory){
13    // call _f with state variables
14    _f(arr, map, myStructs[1]);
15    // get a struct from a mapping
16    MyStruct storage myStruct = myStructs[1];
17    myStruct.foo = 4;
18    // create a struct in memory
19    MyStruct memory myMemStruct = MyStruct(0);
20    MyStruct memory myMemStruct2 = myMemStruct;
21    myMemStruct2.foo = 1;
22
23    MyStruct memory myMemStruct3 = myStruct;
24    myMemStruct3.foo = 3;
25    return (myStruct, myMemStruct2, myMemStruct3);
26  }
27
28  function _f(
```

Tut 17:

LEARNETH 2.6.2 Learneth Solidity Beginner Course

10.1 Transactions - Ether and Wei 17 / 19

`wei` is the smallest subunit of *Ether*, named after the cryptographer Wei Dai. *Ether* numbers without a suffix are treated as `wei` (line 7).

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

One `ether` is equal to 1,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

★ Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer **Show answer**

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5   uint public oneWei = 1 wei;
6   // 1 wei is equal to 1
7   bool public isOneWei = 1 wei == 1;
8
9   uint public oneEther = 1 ether;
10  bool public isOneEther = 1 ether == 1e18;
11
12  uint public oneGwei = 1 gwei;
13  bool public isOneGwei = 1 gwei == 1e9;
14 }
```

Tut 18:

LEARNETH

10.2 Transactions - Gas and Gas Price

Gas prices are denoted in gwei.

Gas limit

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about gas on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin `Gas Profiler` to check for the gas cost of transactions.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }
```

Tut 19:

LEARNETH

10.3 Transactions - Sending Ether

If you change the parameter type for the functions `sendViaTransfer` and `sendViaSend` (line 33 and 38) from `payable address` to `address`, you won't be able to use `transfer()` (line 35) or `send()` (line 41).

Watch a video tutorial on Sending Ether.

Assignment

Build a charity contract that receives Ether that can be withdrawn by a beneficiary.

1. Create a contract called `Charity`.
2. Add a public state variable called `owner` of the type `address`.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
51 }
52
53 contract Charity {
54     address public owner;
55
56     constructor() {
57         owner = msg.sender;
58     }
59
60     function donate() public payable {}
61
62     function withdraw() public {
63         uint amount = address(this).balance;
64         (bool sent, bytes memory data) = owner.call{value: amount}("");
65         require(sent, "Failed to send Ether");
66     }
67 }
68 }
```

Conclusion:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.